

Advice for rookie programmers

Vivek Krishnamoorthy

For some of you, this would be the first time you're learning to program. For others, you might have dabbled in it years ago and are out of touch.

The good news is that coding is a very learnable skill. If you put in the time and the effort, you will learn to program. It will be challenging at the outset but hang in there.

As a quant finance educator, I realize it takes more than just attending Python lectures to be able to freely express your ideas in it. The rest of the write-up is a set of recommendations based on my experiences and discussions with students, friends and colleagues.

Pointers before the EPAT Python lectures

1. Practise everyday

Even it's just half an hour. Open your Python terminal. I don't care what you prefer. It could be the Spyder IDE. Or Jupyter Notebook. Or Jupyter Lab. Or the Python/IPython console. Anything really. *One simple way to do this is to take the Notebook file I've shared and mess with the code in it. Change values and see what happens. Experimenting with it will get you accustomed to Python syntax and also help you figure out limitations. All of this will help you learn.* Keep at it, and it'll eventually start getting more enjoyable. You can teach yourself to do almost anything with this method. Whether it's reading and writing in Python, or playing squash or learning Japanese or taming a lion (maybe not the last one).

2. Learn to research topics online

I would like each of you to learn to become independent learners. And to get there, you need to figure out how to look things up online. This is an important life skill especially for the times we live in. The online Python community is incredibly helpful and enthusiastic. The [official Python documentation](#) is regularly updated and useful albeit a bit technical. [Stackoverflow](#) is a fantastic place to look up answers to questions. This could be anything from the most basic to the very advanced. You can be almost certain that any doubts you have, has been asked by someone before and been answered too. Even [Quora](#) can be useful. Or a plain old web search.

3. Make a note of useful ideas

Through your Pythonic journey, in class or otherwise (I encourage you to consume all kinds of resources be it books, online links, youtube videos, etc. The process of learning is quite arbitrary, and the only way to know what helps you learn effectively is by trying different ones on for size.), there'll be code snippets or illustrations you see that appear handy. You may not even understand it entirely, but note it and save it somewhere. Go back to your notes from time to time and look

it over. You'll be surprised at how soon we forget things if we don't keep reviewing them every now and then.

Tips on using code files shared by instructors

1. The code file you have is a living, breathing document. For each of you. What I mean is, **annotate it** with whatever comments you want. Things you forget (trust me, you will. Your future self will thank you). Things instructors mention in class. That link we may casually reference. Maybe stuff you want to emphasize further just so it sticks in your head.
2. EPAT instructors (including me) usually share a Notebook (or other code files) so that you have a touchstone while we go through our class. So you have both the explanations and the code. However, while you learn to program in Python or for that matter any language, **please type each line in**. Let me repeat that. Type every line. I cannot stress this enough. We may resort to copy-paste during the lecture to save time and cover more topics. But when you review your notes, you should type. That's how you'll learn actively. The idea is to train yourself to read, write and understand code. And for this, it's vital that both your fingers and your brain gets used to it.

Additional reads

1. Python code is highly readable because of its [style guide](#). You might find it interesting to glance through it.
2. Once you have spent a few hours playing with Python, you'll probably enjoy reading [these two](#) introductory posts on interpreting errors.